

DOCUMENTACIÓN TÉCNICA

MÓDULO INVENTARIO

Trabajo práctico final – Sistema de gestión de tienda

Instituto Superior de Formación Técnica N.º 179
Tecnicatura Superior en Análisis de Sistemas
Dr. Carlos Pellegrini

ÍNDICE

1. Introducción y responsabilidad ····· 3
2. Dependencias del módulo ····· 3
3. Estructura de datos y representación ····· 3
4. Funcionamiento de las operaciones ····· 3
5. Integración con persistencia y esperas ····· 5
6. Relación con servidor y transacciones ····· 5

1. Introducción y responsabilidad

El módulo BACKEND/inventario.py concentra las operaciones que mantienen el inventario de productos en memoria y coordinan su persistencia en SQLite. Su responsabilidad no es atender directamente las peticiones HTTP ni construir la interfaz: ofrece funciones que otros módulos invocan para cargar, consultar, agregar, modificar, eliminar y sincronizar productos.

El inventario en memoria se representa mediante una lista enlazada ordenada por id_producto. Cada elemento de esa lista contiene un diccionario con los datos de un producto. El módulo utiliza las primitivas del TDA Lista para operar sobre los nodos, evitando manipular directamente los enlaces.

2. Dependencias del módulo

Dependencia	Uso dentro de inventario.py
BD.persistencia	Lee, inserta, modifica, elimina y consulta registros de productos en SQLite.
BACKEND.esperas	Prepara la notificación al primer cliente en espera y confirma su atención cuando se repone stock.
BACKEND.producto.crear_producto	Construye y valida los diccionarios de producto antes de persistirlos.
BACKEND.tad_listas	Proporciona Lista, barrido, buscar1, inserta1 y eliminar1 para administrar la lista enlazada.

Dependencia directa: inventario.py depende de producto.py, persistencia.py, esperas.py y tad_listas.py. No depende de recomendaciones.py ni de transacciones.py. Esta separación evita que el módulo de inventario conozca la lógica de popularidad o el historial de compras.

3. Estructura de datos y representación

Cada producto se representa como un diccionario con cuatro campos: id_producto, nombre, cantidad_disponible y precio. La lista enlazada mantiene los productos ordenados ascendentemente por id_producto. La ordenación se consigue indicando ese campo en las primitivas inserta1, buscar1 y eliminar1.

```
{"id_producto": 12, "nombre": "Teclado", "cantidad_disponible": 8, "precio": 15000.0}
```

La lista enlazada es la estructura de trabajo del backend durante la ejecución. SQLite es la fuente persistente: si el servidor se detiene, la lista desaparece de memoria, pero los registros almacenados permanecen en la base de datos. Al iniciar nuevamente, cargar_inventario reconstruye la lista.

4. Funcionamiento de las operaciones

4.1 cargar_inventario()

Crea una instancia vacía de Lista y solicita a persistencia.leer_productos() todos los registros almacenados, ordenados por ID. Para cada registro llama a crear_producto(), que vuelve a validar y normalizar los datos. Luego inserta el diccionario en la lista mediante inserta1(inventario, producto, 'id_producto'). Devuelve la lista completa.

Si no hay productos, devuelve una lista vacía. Si un registro persistido contiene datos inválidos, la validación produce ValueError; el módulo no oculta el problema descartando silenciosamente ese registro.

4.2 consultar_productos(inventario)

Utiliza barrido() para obtener los elementos de la lista y devuelve una nueva lista de diccionarios copiados mediante copy(). Las copias impiden que el consumidor modifique accidentalmente los diccionarios internos del inventario. El resultado está ordenado por ID y puede convertirse a JSON para responder a la interfaz.

4.3 consultar_producto(inventario, id_producto)

Primero exige que el identificador sea exactamente un entero positivo. Luego busca un nodo con buscar1(), comparando el campo id_producto. Si no existe, devuelve None. Si existe, devuelve una copia de nodo.info. La función no expone el diccionario interno.

4.4 agregar_producto(...)

Recibe nombre, cantidad disponible y precio. Como el ID definitivo lo genera SQLite, primero llama a crear_producto() con un ID provisional igual a 1, únicamente para validar los demás campos. Si la validación falla, no se escribe nada.

1. Llama a persistencia.insertar_producto() con nombre, stock y precio. SQLite genera el identificador definitivo y lo devuelve.
2. Reemplaza el ID provisional por el ID generado.
3. Inserta el producto validado en la lista enlazada, ordenada por ID.
4. Devuelve una copia del producto creado.

El orden es importante: la lista solo se modifica después de que SQLite confirma la inserción. Así se evita mantener en memoria un producto que no llegó a guardarse.

4.5 modificar_producto(...)

Busca primero el producto actual. Si no existe, devuelve None. Después crea y valida un nuevo diccionario con los datos recibidos. Compara el stock anterior con el nuevo para detectar una reposición desde cero unidades a una cantidad positiva.

Si se produjo esa transición y se recibió el parámetro opcional colas, prepara una notificación para el primer cliente de la cola mediante preparar_notificacion(). La preparación no retira todavía al cliente.

A continuación solicita a persistencia.modificar_producto() que actualice el registro. Si corresponde, también le pasa el ID de la espera que debe atenderse. La persistencia realiza la modificación y la atención de la espera en una misma transacción SQLite. Si el producto no existe en la base, el inventario no se altera.

Una vez confirmada la operación, elimina el elemento anterior con eliminar1() e inserta el nuevo diccionario con inserta1(). Si había una notificación preparada, confirma la atención en la cola mediante confirmar_atencion(). Devuelve una copia del producto actualizado y, cuando corresponde, incorpora el campo notificacion a la respuesta.

4.6 eliminar_producto(inventario, id_producto)

Consulta primero el producto. Si no existe, devuelve None. Luego intenta eliminarlo de SQLite. La operación puede fallar por restricciones de claves foráneas cuando el producto está relacionado con compras o esperas. En ese caso la excepción se propaga y la lista permanece intacta.

Solo si SQLite confirma la eliminación, el módulo retira el nodo mediante eliminar1() y devuelve una copia del producto eliminado. Este orden mantiene sincronizadas la base y la lista.

4.7 sincronizar_productos(inventario, ids_productos)

Recorre los identificadores recibidos y consulta cada producto directamente en persistencia.consultar_producto(). Si falta alguno, lanza ValueError. Si existe, elimina la versión que está en memoria y vuelve a insertarla con los datos actuales de SQLite. Esta función se utiliza después de operaciones de compra o reversión que modifican stock dentro de una transacción.

5. Integración con persistencia y esperas

La capa de persistencia se encarga de ejecutar SQL y confirmar los cambios. Inventario no construye consultas SQL: llama a funciones de alto nivel como leer_productos(), insertar_producto(), modificar_producto(), eliminar_producto() y consultar_producto(). Esta división permite cambiar detalles de almacenamiento sin reescribir las operaciones de la lista.

La integración con esperas aparece principalmente al modificar un producto. Cuando el stock pasa de cero a positivo, inventario prepara la atención del primer cliente y coordina la confirmación de la cola una vez que SQLite guardó el cambio. La persistencia comprueba que la espera indicada siga siendo la primera y registra el aviso junto con la modificación.

6. Relación con servidor y transacciones

servidor.py es el punto de entrada de las operaciones del inventario desde HTTP. Al iniciar, llama a cargar_inventario(). En las rutas de API consulta productos, agrega, modifica o elimina, y utiliza un Lock compartido para proteger las estructuras de memoria frente a solicitudes simultáneas.

transacciones.py utiliza consultar_producto() para validar los productos de una compra y sincronizar_productos() después de que la compra o su reversión se confirma en SQLite. La compra descuenta stock en persistencia; luego inventario actualiza la lista en memoria con el estado definitivo.

```
servidor.py → inventario.py → producto.py / persistencia.py / esperas.py / tad_listas.py
transacciones.py → inventario.py → persistencia.py
```